

Aim: Write a Python program to plot a few activation functions that are being used in neural networks.

Objective: To learn about activation functions and perform its code in python.

What are Activation Functions?

Activation functions are mathematical functions used in Artificial Neural Networks (ANN). They decide whether a neuron should be activated or not by converting the input signal into an output signal. These functions introduce non-linearity into the neural network, which helps the model learn complex patterns from the data.

Why are Activation Functions Used?

1. Introduce Non-Linearity
2. Improve Learning Capability
3. Normalize Output Values
4. Support Backpropagation

Types of Activation Functions

1. Linear Activation Function

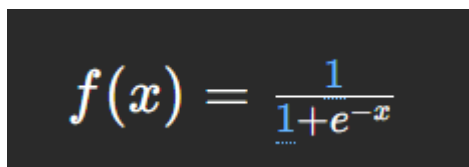
The linear activation function returns the same value as the input. It does not modify the input value.

$$f(x)=x$$

Characteristics: 1. Output range: $-\infty$ to $+\infty$ 2. Simple and easy to compute 3. Mainly used in regression problems

2. Sigmoid Activation Function

The sigmoid function converts input values into the range 0 to 1. It produces an S-shaped curve.


$$f(x) = \frac{1}{1+e^{-x}}$$

Characteristics: 1. Output range: 0 to 1 2. Commonly used in binary classification problems 3. Smooth and differentiable

3. Tanh Activation Function

Tanh (Hyperbolic Tangent) is similar to sigmoid but its output ranges from -1 to 1.

$f(x)=\tanh(x)$ Characteristics:

- Output range: -1 to 1
- Zero-centered output
- Performs better than sigmoid in many cases

4. ReLU Activation Function

ReLU stands for Rectified Linear Unit. It outputs 0 for negative values and the input itself for positive values.

Formula:

$$f(x) = \max(0, x)$$

Characteristics:

- Computationally efficient
 - Helps reduce vanishing gradient problem
 - Most widely used activation function in deep learning
-

5. Softmax Activation Function

Softmax converts input values into probabilities. The sum of all output probabilities is equal to 1.

Formula:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Characteristics:

- Used in multiclass classification problems
- Outputs probability distribution
- Commonly used in the output layer of neural networks

Code for practical

Aim:

Write a Python program to plot a few activation functions used in Neural Networks

Import required libraries

import numpy as np

import matplotlib.pyplot as plt

Generate input values

x = np.linspace(-10, 10, 100)

Linear Activation Function

def linear(x):

return x

Sigmoid Activation Function

def sigmoid(x):

return 1 / (1 + np.exp(-x))

Tanh Activation Function

```
def tanh(x):
```

```
    return np.tanh(x)
```

```
# ReLU Activation Function
```

```
def relu(x):
```

```
    return np.maximum(0, x)
```

```
# Softmax Activation Function
```

```
def softmax(x):
```

```
    exp_x = np.exp(x - np.max(x))
```

```
    return exp_x / np.sum(exp_x)
```

```
# Create subplots
```

```
plt.figure(figsize=(10, 8))
```

```
# Linear Plot
```

```
plt.subplot(3, 2, 1)
```

```
plt.plot(x, linear(x))
```

```
plt.title("Linear")
```

```
plt.grid(True)
```

```
# Sigmoid Plot
```

```
plt.subplot(3, 2, 2)
```

```
plt.plot(x, sigmoid(x))
```

```
plt.title("Sigmoid")
```

```
plt.grid(True)
```

```
# ReLU Plot
```

```
plt.subplot(3, 2, 3)
```

```
plt.plot(x, relu(x))
```

```
plt.title("ReLU")
```

```
plt.grid(True)
```

```
# Tanh Plot
```

```
plt.subplot(3, 2, 4)
```

```
plt.plot(x, tanh(x))
```

```
plt.title("Tanh")
```

```
plt.grid(True)
```

```
# Softmax Plot
```

```
plt.subplot(3, 2, 5)
```

```
plt.plot(x, softmax(x))
```

```
plt.title("Softmax")
```

```
plt.grid(True)
```

```
# Adjust layout
```

```
plt.tight_layout()
```

```
# Display plots
```

```
plt.show()
```

```
# Example Input Array
```

```
arr = np.array([-1, 0, 1])
```

```
# Output of Activation Functions
```

```
print("Input Array:", arr)
```

```
print("Sigmoid Output:")
```

```
print(sigmoid(arr))
```

```
print("Tanh Output:")
```

```
print(tanh(arr))
```

```
print("ReLU Output:")
```

```
print(relu(arr))
```

```
print("Softmax Output:")
```

```
print(softmax(arr))
```